

已验证能力 DAG 与预研方法论

讨论要点草案 (v0)

ascendc 工程

2026-07-15

摘要

本文档汇总近期关于「用数学模型约束预研写码路径、降低 Agent 幻觉」的讨论要点，定位为 docs/research/ 调研草稿，非定稿技术总结，非交付验收依据。核心对象是域无关的**已验证能力图**及其上的**动态派生系统**；以 ML-KEM-1024 kem.keygen 为样例校准，计划用 kem.encaps / kem.decaps 等任务做规则验证实验。方法论允许随专题讨论迭代，不指望一次提炼出最终形态。后续若干日将围绕本方向展开专题讨论，中间可穿插测试与写码。

目录

1 动机与问题陈述	2
1.1 我们实际在做什么	2
1.2 顶层任务的结构直觉	2
1.3 要解决的方法问题	2
2 方法立场（迭代原则）	2
3 对象模型（v0）	3
3.1 节点（能力事实）	3
3.2 边的三种类型（必须分清）	3
3.3 证书状态	3
3.4 分层示意（以 KEM.KeyGen 为样例，非源码树）	3
4 合法性规则（Agent 门禁 v0）	4
5 任务工作流（四步）	4
6 与形式语言、自动机的关联	4
6.1 为何不止于「画一张 DAG」	4
6.2 语言侧（草图）	4
6.3 自动机侧（草图）	5
6.4 相对古典「只增证明堆」的差异点	5
7 域无关性：不只服务 ML-KEM	5

目录	2
8 样例校准: kem.keygen	6
9 验证实验计划	6
10 建议的深挖日程 (M0–M5)	7
11 刻意未决议事项	7
12 立论草案 (一句话)	7
13 文档地位与后续	7

1 动机与问题陈述

1.1 我们实际在做什么

在 KEM KeyGen、PKE KeyGen / 加解密等路径上，开发方式是**层层递进**的：先做最底层单点技术实验，再在已通过的能力上拓展计算任务。一部分能力被视为「当前不可动摇的底层事实」——例如 NTT、内积、哈希——而其下还有更底层的平台事实，例如 DataCopy、向量加、矩阵乘与 tiling、同步壳等。后续进展默认建立在这些事实已经正确验证过的前提上。

1.2 顶层任务的结构直觉

顶端的 KEM 类算子，既像树状结构的根（从顶向下拆依赖），又像 DAG 的汇点（多条已验证中间能力汇入同一目标）。共享中间节点（如 NTT、PKE Encrypt）使结构更接近**有向无环图**而非树。

1.3 要解决的方法问题

能否构造一套数学模型与门禁规则，使 Agent 在开发复杂计算任务（例如未来的 ML-DSA，或其他非密码预研）时：

- 只从已验证底层事实，以及由它们搭建并获证的中间节点出发；
- 高效完成更复杂任务的实现规划与落地；
- 从机制上压缩「跳层发明、未证书边、看似自洽却无验收」的幻觉空间。

目标是形成可指导**一般预研开发**的方法论，而不是仅服务 ML-KEM / ML-DSA 实现。

2 方法立场（迭代原则）

原则	含义
描述先于实现	任务启动时先产出「依赖闭包 + 缺项」，再写码
图服务门禁，不替代标准	DAG / 自动机约束「从哪砌砖」；FIPS / liboqs 等仍是语义 oracle
每验证一条边可改规则	Encaps / Decaps 等实验若暴露漏洞，优先改方法论
粒度：可独立验收块	不必细到每个 API；过细则不可维护
逐渐清晰	允许草案演进；不一次钉死 Rule / Skill

与仓库现有实践的关系：baseline-registry、活跃 INDEX、frozen 判决、customspec 门禁，已经在强制「golden / 实现路径只能引用已登记或已判决可继任的节点」。本稿试图把隐性 DAG **显式化**，并补上「动态证书」与「合法派生」一层。

3 对象模型 (v0)

3.1 节点 (能力事实)

每个节点建议至少携带:

字段	说明
id	稳定名, 如 <code>pke.keygen.k4</code> 、 <code>prim.ntt.tag5t.polybatch</code>
layer	P0 平台公理 / P1 密码 (或领域) 原语 / P2 构件 / P3 顶层算子
iface	I/O、dtype、形状/布局、同步接缝; 可复用的是 iface, 不是源码拷贝
cert	证书: 路径 + CPU+SIM (及交付级 KAT) + commit / 日期
forbid	否定约束 (如某阶段禁 Gather; 禁 Host 密码; 禁从 frozen 抄实现)
repo	权威锚点: <code>ascendc-tests/...</code> 或 <code>examples/stable/...</code>

3.2 边的三种类型 (必须分清)

边类型	含义	KeyGen 例
语义依赖	正确性依赖下游 iface	NTT $\rightarrow$ PKE.KeyGen
接缝 (compose)	A 、 B 各自正确, $A \circ B$ 仍须单独成节点获证	<code>prep</code> $\parallel$ <code>compute</code> $\rightarrow$ <code>device-keygen</code>
否定边	显式禁止的依赖或模板来源	<code>frozen</code> 判决 $\nrightarrow$ 活跃实现

接缝不免费: 组合正确性不能由子节点证书自动推出。这与仓库中「探针 $\rightarrow$ 集成 $\rightarrow$ stable」的实际翻车位置一致。

3.3 证书状态

建议状态机 (可再精简):

`unproven` $\rightarrow$ `probe` $\rightarrow$ `lemma` $\rightarrow$ `deliverable`; `dirty` (上游 iface / tiling 变更后下游失效) .

lemma: 可被多父节点复用; deliverable: stable + registry 级; dirty: 使引理库**非单调** (可收缩), 需进入动态机, 而非静态证明堆。

3.4 分层示意 (以 KEM.KeyGen 为样例, 非源码树)

P0	DataCopy / 向量算子 / MMAD+tiling / PIPE 同步壳
P1	NTT(poly-batch) $\cdot$ basemul/dot $\cdot$ Hash/SHAKE $\cdot$ CBD $\cdot$ ByteEncode ...
P2	Alg.13 PKE.KeyGen (及内部接缝: $\bar{A}$ <code>prep</code> 、 <code>se sample</code> 、 <code>compute</code> ...)
P3	Alg.19 KEM.KeyGen = $G(d,z)$ + Alg.13 + dk 展开 + 生产 I/O 契约

4 合法性规则 (Agent 门禁 v0)

接到目标 T (如 `kem.encaps`) 时, 只允许:

1. **展开** T 的依赖闭包 (语义边 + 接缝边)。
2. **分类**: 已有 `lemma/deliverable`; 缺项; 仅有否定边或 `frozen`。
3. **缺项** $\Rightarrow$ 新建探针或 `exp`, **经验证写入图**后再被 T 引用; 禁止在 T 任务中「顺便发明」未登记的 `P0/P1`。
4. **组合**只通过已声明接缝; 新接缝 = 新节点, 须认证。
5. **验收**: T 的 I/O 与 `golden / KAT` 一致; 禁止以「与参考源码同构」为证。
6. **自包含**: 实现挂 `iface` 契约, 不跨目录偷源码 (`library/shared` 等仓库约定除外); 图边 $\neq$ `#include` 路径。

一句话: 只能沿已认证边生长; 幻觉 $\approx$ 画出一条没有证书的边。

5 任务 workflow (四步)

1. 目标声明 $T = \text{Alg.xx}$, I/O 与验收级别 (`probe / lemma / deliverable`)
2. 闭包展开 列出 `P0--P3`; 标 `ok / missing / dirty`; 写出接缝清单
3. 最小补洞 只补 `missing` 与新接缝 (先探针再集成); 否定边写入 `forbid`
4. 闭环写回 新节点入图; 失败则改规则或封路线, 不默认可复用

`customspec / STATUS / baseline-registry` 可视作图的**文书形态**; 方法论要求: Agent 先交闭包表, 再写码。

6 与形式语言、自动机的关联

6.1 为何不止于「画一张 DAG」

静态 DAG 适合描述**已完成**引理库的依赖投影。预研过程是动态的: 证书增减、接缝认证、`frozen` 抑制、`dirty` 失效传播。因此需要同时借用:

- **形式语言**: 什么算合法的「实现计划 / 任务展开」;
- **自动机 / 工作流**: 什么配置转移合法, 何谓接受与拒绝。

6.2 语言侧 (草图)

取字母表 $\Sigma =$ 「已认证叶子 `iface` 的调用」+ 有限组合子 (分核、分 `launch`、`workspace` 布局、`host` 调度等)。则:

L 未必是正则语言: 嵌套目标、接缝、属性 (形状 / `dtype`) 更接近上下文无关或**属性文法**, 或判断式 $\Gamma \vdash T$ 。

语言概念	预研侧	作用
字 $w \in \Sigma^*$	一条实现计划 / 展开轨迹	可检查、可归档
语言 L	全体合法计划	「不幻觉」 $\approx$ 不产出 L 外的字
产生式	目标改写: $T \Rightarrow u_1 \cdots u_k$	约束如何拆任务
成员判定	对闭包表做合法派生检查	动手前拦截
禁字 / 否定信息	frozen、forbid、未登记块	显式「不可说」

6.3 自动机侧（草图）

配置

$$q = (\Gamma, F, S),$$

其中 Γ 为已证书节点集, F 为前沿目标, S 为接缝与约束 (含 forbid)。候选转移:

转移	含义
Expand	将 $T \in F$ 按产生式拆成子目标
Probe	为缺项建探针任务 (扩展工作, 不立刻进 Γ)
Certify	验收通过 $\Rightarrow$ 节点入 Γ
Compose	仅当子节点均在 Γ 时, 接缝节点可认证
Dirty	上游 iface 变 $\Rightarrow$ 下游退出 Γ
Freeze	加入否定: 禁止再走某边

- **接受**: 目标 $T^* \in \Gamma$ 且生产 I/O 契约满足;
- **拒绝阱**: 使用未证书边、frozen 边、跳层发明终端。

可工作叙事 (待专题讨论 refinement):

预研方法论 = 定义配置、合法转移与接受条件;

静态 DAG 是引理库 Γ 的依赖投影;

Agent 只在该自动机上搜索接受路径; 幻觉定义为非法转移。

6.4 相对古典「只增证明堆」的差异点

dirty 使 Γ 可收缩。这一非单调性应视为方法论特征, 而非瑕疵: 证书绑定具体 iface 与实现锚点, 上游变更必须传播失效。

7 域无关性：不只服务 ML-KEM

抽出密码语义后, 四类对象仍成立:

同一套问题在非密码预研同样出现: 终端从哪来、组合是否免费、失败如何进禁止规则、Agent 是整算子生成还是合法转移搜索、进度如何用 $|\Gamma|$ / dirty 波及度量。ML-KEM 因其 oracle 清晰、接缝多、frozen 否定边丰富, 适合作压力测试床, 而不是方法论边界。

对象	一般预研含义
终端 (axiom)	本任务不再证明的底层事实 (探针、平台行为、外部 oracle)
非终端 (goal)	待构造能力
产生式 (compose)	如何用已有能力拼出新能力；接缝是一等公民
证书 (judgment)	$\Gamma \vdash u : \text{cert}$

8 样例校准：kem.keygen

用已交付的 ML-KEM-1024 KEM KeyGen (stable + device 探针族) 校准规则是否够用：

检查项	问题
分层是否干净	P3 是否只依赖已证书 P2，而非直接重写 NTT 等 P1
接缝是否显式	2-launch、Alg.16 尾、dk 展开等是否各自有证或明确归属
registry 是否对齐	golden 计算块 $\leftrightarrow$ 图上 lemma，有无旁路未登记内核
否定边是否写入	Host 禁密码默认路径、禁移植参考实现为 AscendC 规格

缺口记为**方法论债**，进入专题讨论修改 schema；不假装图已完备。

仓库锚点（讨论时查阅，不在此复述实现）：

- `examples/stable/stable-fips203-mlkem-kem-keygen-k4/`
- `docs/specs/fips203-mlkem1024-kem-keygen-baseline-registry.md`
- `ascendc-tests/pass-fix-f203-alg19-kem-keygen-device-k4/` 等活跃探针

9 验证实验计划

实验	目的	期望学到什么
A. Encaps	用 v0 规则做前瞻闭包	是否强制挂已证 PKE.Encrypt / Hash，而非 correctness vendor；漏算接缝否
B. Decaps	多父 DAG	同时依赖 Encrypt+Decrypt+KeyGen I/O；失败路径是否成独立接缝
C. 对照失败	故意跳层描写	规则能否在动手前拦截
D. 第二域	非 KEM 预研链套同一机	证明不是密码专用话术

成功标准**不是**「任务一次写对」，而是：偏差能被归因到「缺边 / 假边 / 脏证书」之一，并**回写规则**。

近期工程穿插：WSL / 探针侧 Encaps device 等写码与测试可作为实验 A 的素材，但本专题讨论焦点仍是方法论规则，而非单次用例 PASS。

10 建议的深挖日程 (M0–M5)

M0 词汇表: axiom / lemma / seam / forbid / dirty / derive (域无关)。

M1 用 kem.keygen 事后重放一段合法派生轨迹 (纸面即可)。

M2 写出 Encaps / Decaps 的 q_0 与允许转移; 标缺项。

M3 成员检查: 对闭包表做合法/非法判定; 故意非法计划应被拒。

M4 第二域试套 (如纯 tiling / 同步探针晋级链)。

M5 结论稳定后, 按写作模板迁入 docs/notes/; 是否挂钩 Rule/Skill 另议。

刻意顺序: KeyGen 重放 (校准概念) → Encaps 前瞻 (校准动态) → 换域 (校准一般性) → 再谈工具化。

11 刻意未决议事项

- 节点是否 1:1 对应仓库目录名;
- 图用 Markdown 表还是机器可读 YAML / JSON;
- 何时写入 Rule / Skill (建议至少 Encaps+Decaps 各一轮后再谈);
- 形式化深度: 工作流网 / Petri 网 / 属性文法 / 判断式, 选哪一种作为主叙事;
- 完整可判定性与复杂度 (除非单开理论主线, 否则暂不作为预研交付门槛)。

12 立论草案 (一句话)

预研开发的合法性, 可刻画为: 在「已证书能力」生成的语言 L 上, 由配置自动机执行 Expand/Probe/Certify/Compose/Dirty/Freeze; 静态 DAG 是引理库 Γ 的依赖投影, 不是全过程。Agent 的职责是在该自动机上搜索接受路径; 幻觉被定义为非法转移。

13 文档地位与后续

- 路径: docs/research/ ——调研草稿, 可废弃、可改写。
- 本文件记录的是方法论讨论要点, 不记录题外话的保护策略讨论。
- 专题讨论期间允许穿插测试与写码; 结论晋级 notes/ 前, 不以本 PDF 约束交付验收。
- 维护: 增删条目同步 docs/research/INDEX.md; 重大决策同步当日 qa/。